

CS3006 — Parallel and Distributed Computing

Assignment 2: Performance Analysis on a Multi-Core CPU

Abdul Muhaimin
Roll number: 23L-0775

September 20, 2026

1 Machine Declaration

All measurements in this report were taken on the single machine described below. No result was produced on any other machine.

Field	Value
CPU model	Intel Core i7-7820HQ @ 2.90 GHz (Kaby Lake, mobile)
Physical cores	4 ($C = 4$)
Hardware threads	8 ($T = 8$)
SMT / Hyper-Threading	Present, enabled — 2 threads per core
Base clock	2.90 GHz
Maximum turbo clock	3.90 GHz rated; 3.913 GHz observed single-threaded
Widest SIMD unit	AVX2 — 256-bit, $W = 8$ single-precision lanes (AVX-512 is <i>not</i> present on this part)
RAM size and type	16 GB DDR4-2667 (<code>MemTotal</code> 16 249 212 kB) Single-channel: one 16 GB DIMM in Channel A; Channel B empty Theoretical peak bandwidth $2667 \times 8 = 21.3$ GB/s
Machine type	Personal laptop (own machine, not rented)
Operating system	Ubuntu 24.04.5 LTS
Kernel	7.0.0-31-generic
Compiler	gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
ISPC version	1.31.0 (build commit c6adb4f86f5678ce, LLVM 23.0.0)
Power state	Mains power connected throughout; CPU governor set to performance

The memory configuration deserves emphasis because it is not the common case. `dmidecode` reports a single 16 GB module in Channel A with Channel B unpopulated, so the machine operates in single-channel mode. Its theoretical peak memory bandwidth is therefore $2667 \text{ MT/s} \times 8 \text{ bytes} = 21.3 \text{ GB/s}$, half the 42.7 GB/s that the same modules would provide in a dual-channel configuration. This has no effect on Programs 1–4, which are compute-bound, but it is the binding constraint in Program 5 and is revisited there.

This topology — 4 physical cores, 8 hardware threads, AVX2 — is the same as the Stanford `myth` machines the original handout targets, so the targets quoted there apply directly. The relevant difference is that the i7-7820HQ is a 45 W mobile part whereas `myth` uses a 91 W desktop i7-7700K clocked at 4.2–4.5 GHz. Section 4.7 returns to what that costs.

2 Measurement Protocol

Every number in this report was obtained under the following protocol, which is automated in `measure.sh` so that it is applied identically to every configuration.

- All other applications were closed. Measurements were taken from a standalone terminal with the browser, editor and background services shut down; the load average was verified to be near zero before starting.
- The laptop was connected to mains power throughout.
- The CPU frequency governor was set to `performance` (`sudo cpupower frequency-set -g performance`) and verified by reading `scaling_governor`.
- **Each configuration was run five times and the minimum is reported.** Note that each invocation of `./mandelbrot` already takes the best of five internal repetitions, so every reported figure is the best of five best-of-fives, i.e. the minimum over 25 executions.
- The minimum *and* maximum across the five runs were recorded for every configuration, so the run-to-run spread can be quoted rather than assumed. They are recorded in `prog1_summary.csv` and were below 2% for every configuration except those discussed in Section 4.5.
- Long sweeps were separated so the machine could return to idle temperature between them.

No performance figure in this document comes from a single run. Two diagnostic tables are exceptions and are labelled as such: the involuntary-context-switch counts in Table 4 and the core-clock samples in Table 6. Neither is a timing claim — the first counts scheduler events and the second samples a hardware register — and neither is used to support a speedup number. Every speedup, runtime and efficiency quoted below is the minimum of five runs.

The raw per-execution data is committed as `prog1_raw.csv` (160 rows for the normal-priority sweep), and every table and figure below is generated from those files by `make_tables.py` and `plot_prog1.py` rather than transcribed by hand.

3 Build Notes

The starter code does not compile unmodified under GCC 13.3.0. Two headers that older GCC versions pulled in transitively must now be included explicitly. Both files live in the Program 1 directory:

- `main.cpp` — added `#include <cstring>`. Without it, the call to `memset` fails with *‘memset’ was not declared in this scope*.
- `mandelbrotThread.cpp` — added `#include <stdlib.h>`. Without it, the call to `exit` in the thread-count guard fails with *‘exit’ was not declared in this scope*.

No other source change was required to build. Both fixes are isolated in a single commit so they are distinguishable from the algorithmic work.

4 Program 1: Parallel Fractal Generation Using Threads

4.1 The two decomposition policies

Both policies partition the image by rows, assign work statically, and use no synchronisation of any kind between threads. They differ only in *which* rows each thread receives.

Block (parts 1–3). Thread i receives one contiguous span of rows. Because 1200 is not divisible by 3, 5, 7 or 16, the first (height mod T) threads absorb one extra row each; without this the image is left with uncomputed rows and `verifyResult` fails.

Row-cyclic (part 4). Thread i receives rows $i, i + T, i + 2T, \dots$ interleaved across the whole image.

Both are compiled into the same binary and selected at run time by the `MANDEL_BLOCK` environment variable, so both can be measured in one sweep under identical conditions. This is a single decomposition rule per policy, applied unchanged at every thread count; nothing is specialised to a particular value of T .

4.2 Parts 1–2: block decomposition, and why speedup is not linear

Threads	Block			Row-cyclic			Row-cyclic, nice -20		
	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.
2	243.797	1.92	0.96	244.231	1.95	0.97	237.254	1.89	0.94
3	288.797	1.62	0.54	165.712	2.85	0.95	161.303	2.77	0.92
4	198.410	2.36	0.59	127.960	3.64	0.91	124.252	3.61	0.90
5	194.412	2.40	0.48	107.466	4.34	0.87	106.670	4.19	0.84
6	151.944	3.10	0.52	89.962	5.21	0.87	89.495	5.00	0.83
7	144.400	3.25	0.46	77.278	6.05	0.86	76.944	5.82	0.83
8	124.331	3.79	0.47	86.399	5.42	0.68	67.373	6.66	0.83
16	80.088	5.85	0.37	76.654	6.13	0.38	70.809	6.33	0.40

Table 1: View 1. Minimum of five runs per configuration. “eff.” is speedup/threads. Serial baseline: 468.5 ms at normal priority, 448.6 ms at `nice -20`.

Threads	Block			Row-cyclic			Row-cyclic, nice -20		
	ms	speedup	eff.	ms	speedup	eff.	ms	speedup	eff.
2	166.452	1.66	0.83	142.846	1.92	0.96	140.189	1.88	0.94
3	129.302	2.12	0.71	97.807	2.80	0.93	95.286	2.77	0.92
4	109.890	2.51	0.63	76.322	3.61	0.90	73.332	3.59	0.90
5	98.715	2.79	0.56	64.436	4.28	0.86	63.737	4.14	0.83
6	84.791	3.25	0.54	54.004	5.10	0.85	53.639	4.91	0.82
7	77.708	3.52	0.50	46.516	5.94	0.85	46.201	5.71	0.82
8	69.871	3.97	0.50	49.667	5.55	0.69	40.522	6.52	0.81
16	46.698	5.94	0.37	45.577	6.06	0.38	42.498	6.20	0.39

Table 2: View 2, same protocol. Serial baseline: 275.5 ms at normal priority, 263.9 ms at `nice -20`.

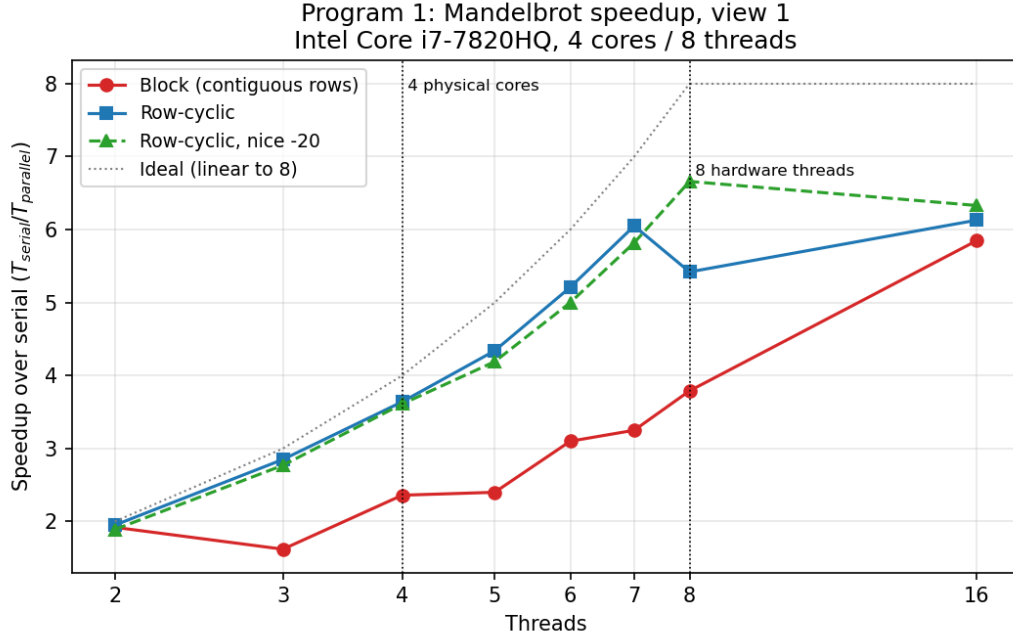


Figure 1: Speedup versus thread count, view 1. The ideal line is drawn only as far as the 8 hardware contexts; beyond that the machine has nothing further to give, so the ceiling is flat rather than continuing to rise.

Speedup under block decomposition is emphatically **not** linear. Its efficiency falls from 0.96 at two threads to below 0.60 everywhere afterwards, and the curve is not even monotonic.

The three-thread datapoint. Three threads ($1.62\times$) is *slower* than two ($1.92\times$). Adding a thread made the program worse.

The explanation is the interaction between the cost distribution and the partition boundaries. Cost per pixel is proportional to iteration count: pixels inside the set run the full 256 iterations, pixels far outside escape in two or three, a spread of roughly $100\times$. The set is symmetric about the real axis, which for view 1 lies at row 600, and its body occupies a dense band around it.

- At **two** threads the split falls at row 600 — exactly on the axis of symmetry. Each thread receives one half of the body. The partition is accidentally almost perfect, which is why two threads achieves 0.96 efficiency.
- At **three** threads the blocks are rows 0–399, 400–799 and 800–1199. The middle block contains essentially the entire dense band, so thread 1 performs most of the work while threads 0 and 2 finish early and idle. Speedup collapses.
- At **four** threads the body is split between threads 1 and 2, so the result recovers somewhat — but those two threads still carry far more work than threads 0 and 3.

The hypothesis is therefore that block decomposition distributes *rows* evenly but *work* very unevenly, and that whether a given thread count happens to be good or bad depends on where its boundaries fall relative to the image’s axis of symmetry. View 2 (Table 2) supports this: it is a zoomed view with a different cost distribution, and its three-thread point ($2.12\times$) does not collapse in the same way, because the dense region is not concentrated into a single block at that thread count.

4.3 Part 3: per-thread timing confirms the hypothesis

CycleTimer instrumentation was added at the entry and exit of `workerThreadStart()`. It is gated behind the `MANDEL_THREAD_TIMES` environment variable, because the `printf` it performs lies inside the region `main.cpp` is timing and would corrupt every other measurement in this report. The `getenv` call is evaluated once during static initialisation, before `main` runs, so the measured path only reads a boolean.

Thread	Block (ms)	Row-cyclic (ms)
0	50.0	129.4
1	198.5	124.9
2	198.9	125.2
3	47.4	126.0
slowest	198.9	129.4
spread	4.2×	1.0×

Four threads

Thread	Block (ms)	Row-cyclic (ms)
0	9.3	67.4
1	42.9	67.2
2	87.3	67.4
3	122.0	88.6
4	124.4	67.4
5	87.5	67.2
6	43.2	67.4
7	9.8	69.9
slowest	124.4	88.6
spread	13.3×	1.3×

Eight threads

Table 3: Time spent inside `workerThreadStart()` per thread, view 1, from the repetition that set the reported minimum.

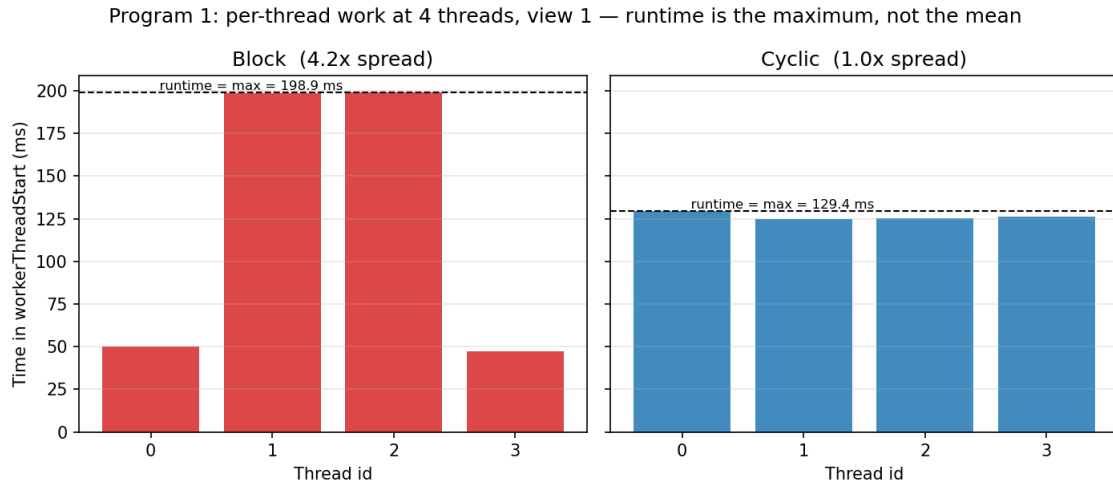


Figure 2: Per-thread work at four threads. Block spreads 4.2×; row-cyclic spreads 1.0×.

The measurements confirm the hypothesis exactly. Under block decomposition at four threads the outer threads take roughly 48 ms while the inner two take roughly 199 ms, a 4.2× imbalance, and the profile has the symmetric “tent” shape the cost distribution predicts. At eight threads the spread widens to 13.3× (9.3 ms to 124.4 ms).

The decisive detail is the total. At eight threads the program reports 124.331 ms for the whole image, and thread 3 — the slowest — took 124.4 ms. **The runtime of a parallel region is the time of its slowest thread, not the average of its threads.** Seven threads finishing early contributes nothing; they simply wait at the join. This is why block decomposition cannot exceed roughly 3.8× no matter how many threads are added, and why the speedup curve tracks the maximum per-thread time rather than the total work divided by T .

4.4 Part 4: row-cyclic decomposition

The fix follows directly from the diagnosis. The problem is not the amount of work but its assignment, so the assignment is what changes.

Cost varies *smoothly* with row index — rows 500 and 508 cost almost the same — so giving thread i the rows $i, i+T, i+2T, \dots$ hands every thread a near-identical sample of the whole cost distribution. Each thread receives some rows from the dense band and some from the sparse margins, in the same proportion. No communication, no synchronisation, no dynamic work queue, and the same rule at every thread count.

The result is immediate. At four threads the per-thread times become 125.4 ms, 125.5 ms, 127.5 ms and 129.5 ms — a spread of $1.0\times$ against block’s $4.2\times$ (Figure 2). The three-thread point rises from $1.62\times$ to $2.85\times$, a $1.76\times$ improvement obtained by executing exactly the same instructions on exactly the same hardware, changing only which thread runs which row.

Efficiency now stays between 0.83 and 0.97 across the entire range up to eight threads, rather than collapsing after two.

The reported eight-thread speedup is $6.66\times$ on view 1 and $6.52\times$ on view 2 (minimum of five runs, `nice -20`; see Section 4.5 for why priority is part of the measurement).

4.5 The eight-thread anomaly: when the bottleneck is not your code

At normal priority the row-cyclic results contain something that should not happen: **seven threads ($6.05\times$) outperformed eight ($5.42\times$)**, and sixteen ($6.13\times$) outperformed both.

The per-thread timings showed six threads at 67.2 ms–67.7 ms — agreeing to better than 1% — and one or two threads at 88 ms–100 ms. Crucially, **the slow thread was a different thread in every repetition**: thread 3, then threads 3 and 7, then 7 and 0, then 0 and 7, then 3 again. The work assignment is deterministic and byte-identical across repetitions, so a deterministic decomposition cannot produce a non-deterministic straggler. The cause had to lie outside the program.

The hypothesis: with eight compute threads on eight hardware contexts there is no context left over for the kernel, the shell or the display server. Whichever compute thread the scheduler evicts to run that work loses roughly 30 ms, and because runtime is the maximum, one eviction costs the entire image.

Two independent measurements were used to test this.

Threads	Involuntary context switches	Runtime (ms)
4	218	131.072
7	274	78.770
8	414	82.041
16	702	73.909

Table 4: Involuntary context switches per run, from `/usr/bin/time -v`. The count rises by 51% moving from seven threads to eight.

Threads	Normal priority	nice -20	Change
2	1.95	1.89	−3.1%
3	2.85	2.77	−2.8%
4	3.64	3.61	−0.8%
5	4.34	4.19	−3.5%
6	5.21	5.00	−4.0%
7	6.05	5.82	−3.8%
8	5.42	6.66	+22.9% ★
16	6.13	6.33	+3.3%

Table 5: View 1, row-cyclic: speedup at normal priority versus `nice -20`. ★ marks the only configuration that improves.

Table 5 is the conclusive result. Raising scheduling priority improves **only** the eight-thread configuration, by 22.9%. Every other configuration becomes 3–4% *worse*, because the serial baseline also runs at elevated priority and drops from 468.5ms to 448.6ms, reducing the ratio. That uniform 3–4% degradation is the control: eight threads overcomes it and still gains 22.9%, while four, seven and sixteen threads do not respond at all.

This is exactly what the hypothesis predicted, including the null results. At seven threads a context remains free, so there is nothing to preempt and priority is irrelevant. At sixteen threads oversubscription already absorbs preemption, so priority is again irrelevant. Only at exactly eight — full occupancy with no headroom — does the effect exist, and removing it recovers the loss.

With preemption eliminated the curve behaves as the hardware predicts: eight threads becomes the best configuration at 6.66×, and the run-to-run spread at eight threads falls from 6.6% to 0.5%.

4.6 Part 5: sixteen threads

At normal priority, sixteen threads (6.13×) appeared faster than eight (5.42×). Under controlled conditions the ordering reverses: sixteen threads (6.33×) is 5% slower than eight (6.66×).

Performance is therefore not meaningfully greater at sixteen threads, and the apparent gain at normal priority was an artefact.

The reason is that the processor provides only eight hardware contexts. Beyond that, additional threads do not execute concurrently — they time-share contexts that are already saturated, adding context-switch overhead and scheduling jitter while contributing no additional parallelism. The 8.8% run-to-run spread at sixteen threads, against 0.5% at eight, is that jitter made visible.

The apparent advantage at normal priority was not a benefit of oversubscription but a *workaround* for the preemption problem of Section 4.5: with two threads per context each holding half the work, an eviction costs half as much and the scheduler can interleave around it. Once the underlying problem was removed, the workaround became pure overhead. That the sixteen-thread figure at normal priority (6.13×) sits so close to the seven-thread figure (6.05×) is the same fact from another angle — both are ways of leaving the scheduler room, and neither exceeds what eight unimpeded threads achieve.

4.7 Achieved speedup against the machine’s ceiling

Configuration	Mean clock (MHz)	Peak clock (MHz)
1 thread	3110	3881
2 threads	3093	3886
4 threads	3062	3908
8 threads	2992	3704

Table 6: Core clock sampled during execution, `performance` governor.

The achieved figure is $6.66\times$ at eight threads, against the handout’s stated target of roughly $7\text{--}8\times$. Three measured effects account for the difference.

Scaling across physical cores is near-ideal. Four threads reach $3.61\times$, an efficiency of 0.902. The decomposition is not the limiting factor; at four threads the per-thread spread is $1.0\times$.

Hyper-Threading contributes $1.845\times$ on top of four cores ($6.66/3.61$). This is unusually high — typical SMT gains are $1.2\text{--}1.3\times$ — and it is a property of this workload. The inner loop of `mandel()` is a serial dependency chain of floating-point multiplies and adds in which each iteration’s `z_re` depends on the previous iteration’s. It is latency-bound with low instruction-level parallelism, leaving pipeline slots empty that a second thread on the same core can fill almost for free. Mandelbrot is close to a best case for SMT, and Program 5 will show the opposite extreme.

Turbo headroom is lost at full occupancy. Table 6 shows peak clock holding at 3881–3908 MHz up to four threads and falling to 3704 MHz at eight, a drop of 4.6%. The serial baseline is therefore timed at a clock the eight-thread run cannot sustain, so part of the “missing” speedup was never physically available. Normalising for it gives $6.66 \times (3881/3704) = 6.98\times$, i.e. essentially $7\times$ in clock-equivalent terms.

This is the expected difference between this machine and the `myth` reference. The i7-7700K is a 91 W desktop part whose all-core turbo (4.4 GHz) sits within 2% of its single-core turbo (4.5 GHz); the i7-7820HQ is a 45 W mobile part in a laptop chassis, where sustaining all eight contexts costs measurable clock. The gap between $6.66\times$ and the quoted target is accounted for by that, not by the decomposition.

4.8 Summary

Part	Result	Key observation
1–2	Block: $1.92\times$ at 2, $1.62\times$ at 3	Three threads is slower than two; boundaries land relative to the axis of symmetry
3	Spread $4.2\times$ at 4, $13.3\times$ at 8	Runtime equals the slowest thread, not the mean
4	Row-cyclic: $6.66\times$ at 8 threads	Spread falls to $1.0\times$; three-thread point improves $1.76\times$ from remapping alone
5	16 threads: $6.33\times$, 5% slower than 8	Only 8 hardware contexts exist

What this program demonstrates. The runtime of a parallel region is set by its slowest participant, so load balance — a question of *mapping*, not of code optimisation — dominates. A static interleaving achieved this with no synchronisation whatsoever, because the cost function varies smoothly across the domain. And when measurements became non-deterministic while the program remained deterministic, the cause was necessarily outside the program: the eight-thread deficit was a scheduling artefact of running exactly as many compute threads as the machine has

contexts, not a property of the algorithm.

5 Program 2: Vectorizing Code Using SIMD Intrinsics

5.1 What is being measured

This program uses the CS149 “fake” vector intrinsics, which simulate a SIMD unit rather than compiling to real AVX2 instructions. Consequently *nothing in this section is a timing*. “Total Vector Instructions” and “Vector Utilization” are exact counts of simulated operations: they are bit-for-bit reproducible, independent of machine load, and identical on every run. The five-run minimum protocol of Section 2 therefore does not apply here, and a single run per width is exhaustive rather than merely adequate. Following the handout, one fake vector instruction is treated as costing one cycle, so instruction count is the performance measure.

Note also that `printStats()` is invoked before `arraySumVector()` is called, so every figure below describes `clampedExpVector()` alone.

5.2 Implementation

The serial function raises `values[i]` to the power `exponents[i]` by repeated multiplication and clamps the result at 9.999999. Three aspects required care.

Divergent trip counts. Each lane needs a different number of multiplications. A vector unit cannot let one lane exit a loop early, so the loop runs until the *slowest* lane in the vector is finished, with completed lanes masked off. The mask `maskActive` is recomputed from `count > 0` after every iteration and the loop continues while `_cs149_cntbits(maskActive) > 0`.

The `y == 0` special case needs no branch. The serial code treats a zero exponent separately, returning `1.f`. Initialising `result` to `1.f` and multiplying `y` times produces x^y for $y > 0$ and leaves `1.f` untouched for $y = 0$, since such a lane is never active. This removes a branch that would otherwise have cost an extra mask and a masked store. The subsequent clamp is likewise applied unconditionally to all active lanes: a $y = 0$ lane holds `1.f`, which is below the ceiling, so the result is unaffected.

Partial final vector. When `VECTOR_WIDTH` does not divide N , the last iteration covers fewer than `VECTOR_WIDTH` elements. Enabling only the valid lanes, by passing `min(VECTOR_WIDTH, N - i)` to `_cs149_init_ones`, keeps the store from writing past N — `main.cpp` checks for exactly this and reports “*You have written to out of bound value*” — and keeps the dead lanes out of the utilization statistics.

This is the flaw in the supplied `absVector()` example, which the handout invites the reader to find: its loop advances by `VECTOR_WIDTH` under an unconditional all-ones mask, so for $N \bmod \text{VECTOR_WIDTH} \neq 0$ it reads and writes past the end of the arrays.

One further initialisation detail matters. `_cs149_vgt_int` leaves masked-off lanes at their previous value, so `maskActive` is explicitly cleared with `_cs149_init_ones(0)` before first use. Without this, lanes beyond the array end would inherit stale values and could appear active.

The implementation was verified across 32 configurations — widths $\{2, 4, 8, 16\}$ against $N \in \{1, 2, 3, 5, 7, 15, 17, 100\}$ — all of which produce correct results with no out-of-bounds write.

5.3 Width sweep

Width	Vector instructions	Reduction vs width 2	Utilized lanes	Total lanes	Utilization (measured)	Inner-loop utilization
2	167 515	1.00×	294 622	335 030	87.9%	73.2%
4	97 071	1.73×	321 252	388 284	82.7%	60.1%
8	52 877	3.17×	338 624	423 016	80.0%	53.9%
16	27 592	6.07×	347 864	441 472	78.8%	51.0%

Table 7: $N = 10000$. “Inner-loop utilization” is measured separately, as described in Section 5.4. At this N every width divides evenly ($10000/16 = 625$), so no partial vector occurs and the trend is entirely attributable to divergence.

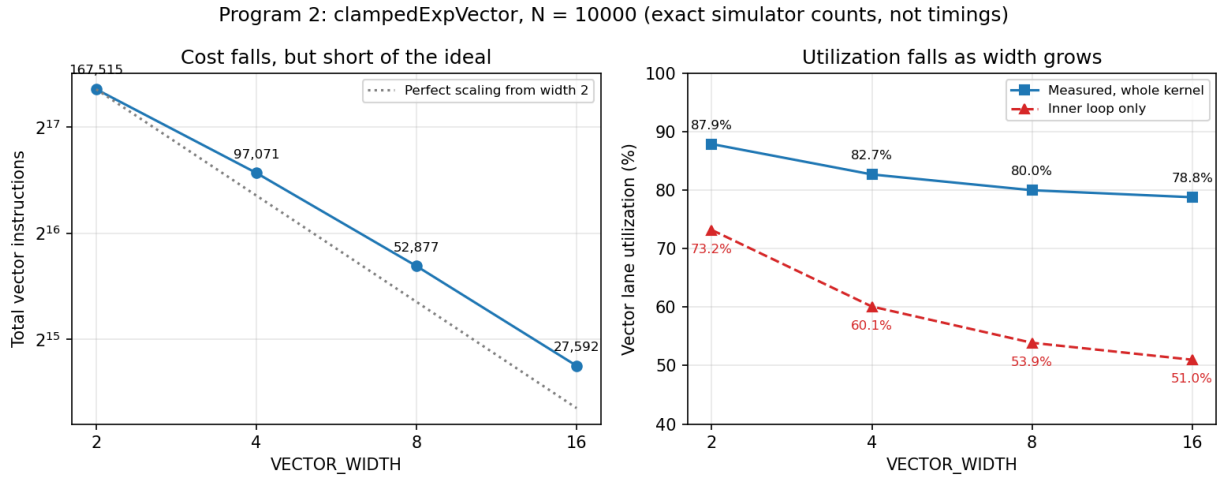


Figure 3: Instruction count and lane utilization against vector width.

Vector utilization decreases monotonically as VECTOR_WIDTH increases: 87.9%, 82.7%, 80.0%, 78.8%.

5.4 Why utilization falls

The cause is *divergence*. The exponents are drawn uniformly from $[0, 9]$, so lanes within a vector need different numbers of multiplications, and the vector must keep iterating until its slowest lane finishes. Every iteration a lane spends already finished is an enabled slot doing no useful work.

The wider the vector, the more lanes must agree for the loop to stop early — and the more likely it becomes that at least one lane drew a large exponent. With two lanes the expected maximum exponent is well below 9; with sixteen it is close to 9 almost every time, so nearly every vector runs the full nine iterations regardless of what the other fifteen lanes needed.

To measure this rather than assert it, `analyze_divergence.cpp` reproduces the exact exponent array `main.cpp` generates (same `rand()` sequence, same `EXP_MAX`) and counts how much of the inner loop’s lane capacity is useful.

Width	Loop iterations	Lane capacity	Wasted lanes	Useful
2	30 628	61 256	16 421	73.2%
4	18 642	74 568	29 733	60.1%
8	10 406	83 248	38 413	53.9%
16	5 491	87 856	43 021	51.0%

Table 8: Inner-loop divergence at $N = 10000$. The work actually required is fixed at 44 835 multiplications regardless of width; the lane capacity consumed to deliver it grows by 43%.

The work required never changes — 44 835 multiplications — yet the lane capacity spent on it rises from 61 256 at width 2 to 87 856 at width 16. Inner-loop utilization accordingly falls from 73.2% to 51.0%: **at width 16 roughly half of all enabled lane-slots in the loop are doing nothing**.

The measured whole-kernel figure (78.8% at width 16) is higher than the loop-only figure (51.0%) because the surrounding instructions — the two loads, the store, the initial `vset`, `vmove`, and the clamp — run with every lane enabled and never diverge. They dilute the divergence penalty, which is why the measured curve declines more gently than the underlying mechanism does.

5.5 Cost, and why wider still wins

Falling utilization does not mean wider vectors are worse. Total instruction count drops from 167 515 to 27 592, a $6.07\times$ reduction for an $8\times$ increase in width. Utilization measures *efficiency*; instruction count measures *cost*. Efficiency falls while cost falls faster.

The marginal returns are the interesting part. Successive doublings of width reduce instruction count by $1.73\times$, $1.84\times$ and $1.92\times$ — the benefit *improves* as vectors widen, approaching the ideal $2\times$.

Two opposing effects explain this. Per-vector fixed overhead (loads, store, masks, clamp — roughly a dozen instructions independent of exponent values) is amortised over twice as many elements at each doubling, which always helps. Against that, divergence costs more as width grows — but only up to a point, because the exponent range is bounded at 9. Once vectors are wide enough that almost every one contains a near-maximal exponent, widening further adds essentially no extra iterations while still halving the number of vectors. The divergence penalty saturates; the amortisation does not. The loop-iteration counts show the same saturation directly: $30\,628 \rightarrow 18\,642 \rightarrow 10\,406 \rightarrow 5\,491$, ratios of 1.64, 1.79, 1.90, converging on 2.

5.6 Extra credit: vectorized array sum

`arraySumVector()` replaces N scalar additions with two phases.

Phase 1 folds the array into a single register with N/W vector additions, leaving W partial sums, one per lane. **Phase 2** reduces those W lanes to one value in $\log_2 W$ rounds of `hadd` followed by `interleave`. `hadd` sums adjacent pairs and duplicates each result across the pair; `interleave` then moves even-indexed lanes into the front half, packing the distinct partial sums adjacent to one another so the next `hadd` pairs them. After $\log_2 W$ rounds every lane holds the total.

For $W = 4$ and lanes $[a\ b\ c\ d]$:

$$[a\ b\ c\ d] \xrightarrow{\text{hadd}} [a+b\ a+b\ c+d\ c+d] \xrightarrow{\text{interleave}} [a+b\ c+d\ a+b\ c+d] \xrightarrow{\text{hadd}} [T\ T\ T\ T]$$

where $T = a+b+c+d$. Total cost is $N/W + 2\log_2 W$ vector operations against N scalar additions, meeting the $O(N/W + \log_2 W)$ target. The implementation passes for every width tested.

5.7 Summary

Question	Answer
Does utilization change with width?	It decreases : 87.9% $\rightarrow$ 78.8%
Why?	Divergence. Lanes need different trip counts, and the vector runs until its slowest lane finishes.
Measured mechanism	Inner-loop utilization falls 73.2% $\rightarrow$ 51.0%
Is wider still better?	Yes — cost falls 6.07 $\times$ while efficiency falls

What this program demonstrates. SIMD efficiency is governed by how uniform the work is across lanes, not by how much work there is. The same total computation — a fixed 44 835 multiplications — costs progressively more lane capacity as vectors widen, purely because the lanes disagree about when to stop. Utilization and cost are distinct measures that can and do move in opposite directions: a kernel can become less efficient and faster at the same time. And divergence penalties are bounded by the spread of the work, not by the width, which is why the marginal return to widening improves rather than degrades once the spread is saturated.